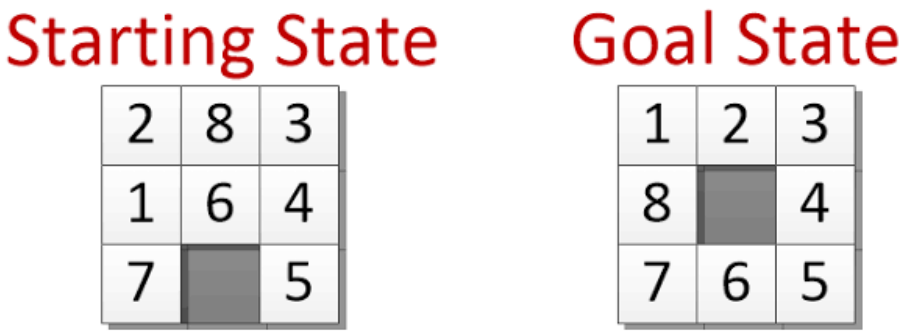
Katie de la Pena

Professor LeBlanc

Artificial Intelligence - COMP255

18 October 2026

8-Puzzle: Program #2 Report



SUMMARY:

The purpose of this Python program is to solve the "8-puzzle" by either using a set board or a randomized board (for experimental timing purposes). The 8-Puzzle is a 3x3 sliding puzzle where there is one empty slot, and to complete the puzzle, the empty slot must be moved in a particular direction and order so that it can reach the "Goal State" ("Looking into 8-Puzzle Heuristics"). In this particular program "0" depicts the blank tile.

Goal State for this Program:

```
-------------
| 0 | 1 | 2 |
-------------
| 3 | 4 | 5 |
-------------
| 6 | 7 | 8 |
-------------
```

This program used two different algorithms: A* search and Best-First Search. Both of these algorithms used two different heuristics: Manhattan Distance and Number of misplaced tiles. In the following sections, these algorithms and heuristics will be described in more detail, and there will be tables and graphs depicting comparisons between these algorithms and heuristics. There will also be examples of sample output the program outputs to the console.

METHODS:

When first calling any of these algorithms, the program initializes a board of a certain heuristic (either "Manhattan" or "Misplaced"), which sets the initial heuristic value. The "Manhattan" heuristic refers to Manhattan Distance, which calculates the total sum of all misplaced tiles necessary minimum paths to their "GOAL" state. For example, if the number 3's shortest path to its spot on the GOAL board is 3, the number 5's shortest path to its spot is 6, and number 4's spot shortest path is 1, the Manhattan Distance would be 10. The "Misplaced" heuristic refers to the amount of misplaced tiles from their desired GOAL location. For example, if 3, 8, and 1 were not in their correct GOAL location, the Misplaced heuristic would be 3.

The two algorithms, as stated above, are A* and Best-First Search. The Best-First function uses the initial board (either set or randomly created) to identify the initial heuristic value based on either Manhattan or Misplaced (depending on the heuristic type passed into the function). This initial board is then put onto a Priority Queue, which evaluates boards based on lowest heuristic score. The initial board is popped off and children boards are created based on possible movements of the blank tile (up, left, right, down). Each child board is assigned a heuristic value and is then put in order onto the priority queue if it has not been visited already.

This process continues until the goal board is reached or the priority queue is empty. Since this search relies on solely the heuristic value of the board, it does not guarantee the shortest solution.

A* search algorithm also uses a very similar method to Best-First search, however, it also calculates the path cost, which is aimed at determining the most efficient solution. The total cost is calculated by adding the number of moves from the initial board to the current board (everytime a tile is moved, 1 is added to the total number of moves) and the heuristic value (Manhattan or Misplaced Tiles). This is again repeated until the goal state is reached or the priority queue is empty.

SAMPLE OUTPUT:

Below is a sample output of an easy board for Best-First Search and A* Search using the Manhattan heuristic:

```
SEARCH ALGORITHM:  Best-First Search
HEURISTIC:  Manhattan
DONE!! Number of moves to solve puzzle:  4

Starting Board
-------------
| 3 | 1 | 2 |
-------------
| 4 | 7 | 5 |
-------------
| 6 | 8 | 0 |
-------------

Move #  1
-------------
| 3 | 1 | 2 |
-------------
| 4 | 7 | 5 |
-------------
| 6 | 0 | 8 |
-------------
```

Move # 2

| 3 | 1 | 2 |

| 4 | 0 | 5 |

| 6 | 7 | 8 |

Move # 3

| 3 | 1 | 2 |

| 0 | 4 | 5 |

| 6 | 7 | 8 |

Move # 4

| 0 | 1 | 2 |

| 3 | 4 | 5 |

| 6 | 7 | 8 |

Time taken: 0.00010 seconds
YES
SEARCH ALGORITHM: A* Search
HEURISTIC: Manhattan
DONE!! Number of moves to solve puzzle: 4

Starting Board

| 3 | 1 | 2 |

| 4 | 7 | 5 |

| 6 | 8 | 0 |

Move # 1

| 3 | 1 | 2 |

| 4 | 7 | 5 |

| 6 | 0 | 8 |

Move # 2

| 3 | 1 | 2 |

| 4 | 0 | 5 |

| 6 | 7 | 8 |

Move # 3

| 3 | 1 | 2 |

| 0 | 4 | 5 |

| 6 | 7 | 8 |

Move # 4

| 0 | 1 | 2 |

| 3 | 4 | 5 |

| 6 | 7 | 8 |

Time taken: 0.00007 seconds
YES

RESULTS:

Algorithm	Average Max Size of Queue	Average Runtime (sec)
Best-First: *h(n) = # of misplaced tiles*	450.23400	0.21695
Best-First: *h(n) = Manhattan Distance*	189.31200	0.03551

Table 1: Comparison of Heuristics for 1000 trials of randomly generated (valid) starting 8-puzzle boards

Board Number Tried	Size of Queue	Size of Visited List	Time (Seconds) So Far
0	0	1	1.66893×10^{-6}
1	1	3	1.66893×10^{-5}
2	2	5	3.29017×10^{-5}
3	4	8	5.38825×10^{-5}
4	5	10	7.08103×10^{-5}

Table 2: Using A* Search and Manhattan Heuristic, the initial lines of output of one solution while solving the 8-puzzle (easy solution). The sizes of the data structures, Queue and Visited List, are shown along with the time (in seconds) as the boards are traversed.

Board Number Tried	Size of Queue	Size of Visited List	Time (Seconds) So Far
0	0	1	9.53674×10^{-7}
1	1	3	1.40666×10^{-5}
2	2	5	2.71797×10^{-5}
3	4	8	4.38690×10^{-5}
4	5	10	5.79357×10^{-5}

Table 3: Using A* Search and Misplaced Tiles Heuristic, the initial lines of output of one solution while solving the 8-puzzle (easy solution). The sizes of the data structures, Queue and Visited List, are shown along with the time (in seconds) as the boards are traversed.

Board Number Tried	Size of Queue	Size of Visited List	Time (Seconds) So Far
0	0	1	4.05311×10^{-6}
1	1	3	4.12461×10^{-5}
2	2	5	6.10351×10^{-5}
3	4	8	8.32080×10^{-5}
4	5	10	0.00010

Table 4: Using Best-First Search and the Manhattan Heuristic, the initial lines of output of one solution while solving the 8-puzzle (easy solution). The sizes of the data structures, Queue and Visited List, are shown along with the time (in seconds) as the boards are traversed.

Board Number Tried	Size of Queue	Size of Visited List	Time (Seconds) So Far
0	0	1	3.81469×10^{-6}
1	1	3	3.09944×10^{-5}
2	2	5	4.79221×10^{-5}
3	4	8	6.77108×10^{-5}
4	5	10	8.17775×10^{-5}

Table 5: Using Best-First Search and the Misplaced Tiles Heuristic, the initial lines of output of one solution while solving the 8-puzzle (easy solution). The sizes of the data structures, Queue and Visited List, are shown along with the time (in seconds) as the boards are traversed.

COMPARISON:

As seen in Table 1, the misplaced tiles heuristic lead to an extremely large number of boards on the Queue in Best-First Search. It also led to a longer average runtime (0.21695) than Best-First Search using Manhattan distance (0.03551). When looking at Tables 2-5, the time to work through the queue list is significantly shorter using the A* search algorithm than the Best-First Search algorithm. When looking at each algorithm individually, the Misplaced Tiles Heuristic appears to have a bit of a shorter search time through this particular easy starting board.